CI/CD

Od commitu k nasazení

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

20.10.2026

CC BY-NC-SA 4.0

Úvod

Známá situace

Vzpomeňte si na odevzdávání domácích úkolů

Známá situace

Vzpomeňte si na odevzdávání domácích úkolů

Chvíli po `git push` se u commitu objeví **zelená** ✓, nebo **červený** křížek

Známá situace

Vzpomeňte si na odevzdávání domácích úkolů

Chvíli po `git push` se u commitu objeví **zelená** ✓, nebo **červený** křížek

Kdo to spouští?

Známá situace

Vzpomeňte si na odevzdávání domácích úkolů

Chvíli po `git push` se u commitu objeví **zelená** ✓, nebo **červený** křížek

Kdo to spouští?

Nikdo – **automat**

Známá situace

Vzpomeňte si na odevzdávání domácích úkolů

Chvíli po `git push` se u commitu objeví **zelená** ✓, nebo **červený** křížek

Kdo to spouští?

Nikdo – **automat**

Dnes se podíváme, jak funguje a jak si takový automat postavit

Známá situace

Jak často podle vás nasazuje Amazon novou verzi do produkce?

Známá situace

Jak často podle vás nasazuje Amazon novou verzi do produkce?

Přibližně **každých 12 sekund**

Známá situace

Jak často podle vás nasazuje Amazon novou verzi do produkce?

Přibližně **každých 12 sekund**

Bez automatizace by to nešlo – ručně by jedno nasazení trvalo déle, než než přijde další

Známá situace

Jak často podle vás nasazuje Amazon novou verzi do produkce?

Přibližně **každých 12 sekund**

Bez automatizace by to nešlo – ručně by jedno nasazení trvalo déle, než než přijde další

Pozn.: Zdroj: Jon Jenkins (Amazon), konference Velocity 2011 – řádový odhad, dnes je číslo ještě vyšší

Ruční nasazení

Jak byste **bez automatizace** dostali novou verzi aplikace k uživatelům?

Ruční nasazení

Jak byste **bez automatizace** dostali novou verzi aplikace k uživatelům?

1. Spustit testy na svém počítači

Ruční nasazení

Jak byste **bez automatizace** dostali novou verzi aplikace k uživatelům?

1. Spustit testy na svém počítači
2. Přeložit TypeScript (`npm run build`)

Ruční nasazení

Jak byste **bez automatizace** dostali novou verzi aplikace k uživatelům?

1. Spustit testy na svém počítači
2. Přeložit TypeScript (`npm run build`)
3. Zkopírovat soubory na server

Ruční nasazení

Jak byste **bez automatizace** dostali novou verzi aplikace k uživatelům?

1. Spustit testy na svém počítači
2. Přeložit TypeScript (`npm run build`)
3. Zkopírovat soubory na server
4. Restartovat aplikaci

Ruční nasazení

Jak byste **bez automatizace** dostali novou verzi aplikace k uživatelům?

1. Spustit testy na svém počítači
2. Přeložit TypeScript (`npm run build`)
3. Zkopírovat soubory na server
4. Restartovat aplikaci

Co všechno se může pokazit?

Ruční nasazení

Ruční nasazení

- Na testy se **zapomene** – „je to jen malá změna“

Ruční nasazení

- Na testy se **zapomene** – „je to jen malá změna“
- Na serveru je **jiná verze Node.js** než u vás

Ruční nasazení

- Na testy se **zapomene** – „je to jen malá změna“
- Na serveru je **jiná verze Node.js** než u vás
- Zkopíruje se **stará verze** souborů

Ruční nasazení

- Na testy se **zapomene** – „je to jen malá změna“
- Na serveru je **jiná verze Node.js** než u vás
- Zkopíruje se **stará verze** souborů
- „Funguje mi to lokálně“ – ale nikde jinde

Ruční nasazení

- Na testy se **zapomene** – „je to jen malá změna“
- Na serveru je **jiná verze Node.js** než u vás
- Zkopíruje se **stará verze** souborů
- „Funguje mi to lokálně“ – ale nikde jinde

Lidé zapomínají a dělají chyby

Ruční nasazení

- Na testy se **zapomene** – „je to jen malá změna“
- Na serveru je **jiná verze Node.js** než u vás
- Zkopíruje se **stará verze** souborů
- „Funguje mi to lokálně“ – ale nikde jinde

Lidé zapomínají a dělají chyby – **automaty ne**

Continuous Integration

Definice CI

Continuous Integration (průběžná integrace, CI) je

Definice CI

Continuous Integration (průběžná integrace, CI) je praxe, kdy vývojáři **často** slučují změny do společné větve

Definice CI

Continuous Integration (průběžná integrace, CI) je praxe, kdy vývojáři **často** slučují změny do společné větve a každou změnu **automaticky ověří stroj**

Definice CI

Continuous Integration (průběžná integrace, CI) je praxe, kdy vývojáři **často** slučují změny do společné větve a každou změnu **automaticky ověří stroj**

- Pojem zavedl Grady Booch (1991)

Definice CI

Continuous Integration (průběžná integrace, CI) je praxe, kdy vývojáři **často** slučují změny do společné větve a každou změnu **automaticky ověří stroj**

- Pojem zavedl Grady Booch (1991)
- Popularizoval ho extrémní programování (Kent Beck, konec 90. let)

Definice CI

Continuous Integration (průběžná integrace, CI) je praxe, kdy vývojáři **často** slučují změny do společné větve a každou změnu **automaticky ověří stroj**

- Pojem zavedl Grady Booch (1991)
- Popularizoval ho extrémní programování (Kent Beck, konec 90. let)
- Dnes je to **standard** – setkáte se s ním v každé firmě

Definice CI

Jakou analogii si můžete představit?

Definice CI

Jakou analogii si můžete představit?

Kontrolu kvality na **výrobní lince**

Definice CI

Jakou analogii si můžete představit?

Kontrolu kvality na **výrobní lince**

- Každý výrobek projde stejnou sadou kontrol

Definice CI

Jakou analogii si můžete představit?

Kontrolu kvality na **výrobní lince**

- Každý výrobek projde stejnou sadou kontrol
- Vadný kus linka zastaví **hned**

Definice CI

Jakou analogii si můžete představit?

Kontrolu kvality na **výrobní lince**

- Každý výrobek projde stejnou sadou kontrol
- Vadný kus linka zastaví **hned**
- Ne až u zákazníka

Automatické kontroly

Co by měl automat na každém commitu zkontrolovat?

Automatické kontroly

Co by měl automat na každém commitu zkontrolovat?

- **Lint** – dodržuje kód pravidla? (ESLint)

Automatické kontroly

Co by měl automat na každém commitu zkontrolovat?

- **Lint** – dodržuje kód pravidla? (ESLint)
- **Testy** – funguje to, co fungovalo včera?

Automatické kontroly

Co by měl automat na každém commitu zkontrolovat?

- **Lint** – dodržuje kód pravidla? (ESLint)
- **Testy** – funguje to, co fungovalo včera?
- **Build** – jde projekt vůbec přeložit? (`tsc`)

Automatické kontroly

Co by měl automat na každém commitu zkontrolovat?

- **Lint** – dodržuje kód pravidla? (ESLint)
- **Testy** – funguje to, co fungovalo včera?
- **Build** – jde projekt vůbec přeložit? (`tsc`)
- **Závislosti** – nemají známé zranitelnosti? (`npm audit`)

Automatické kontroly

Co by měl automat na každém commitu zkontrolovat?

- **Lint** – dodržuje kód pravidla? (ESLint)
- **Testy** – funguje to, co fungovalo včera?
- **Build** – jde projekt vůbec přeložit? (`tsc`)
- **Závislosti** – nemají známé zranitelnosti? (`npm audit`)

Přesně tuhle sadu kontrol nazýváme **pipeline**

GitLab CI

První pipeline

Kde se pipeline definuje?

První pipeline

Kde se pipeline definuje?

V souboru `.gitlab-ci.yml` v **kořeni repozitáře**

První pipeline

Kde se pipeline definuje?

V souboru `.gitlab-ci.yml` v **kořeni repozitáře**

```yaml
test:
  image: node:24
  script:
    - npm ci
    - npm test
```

První pipeline

Kde se pipeline definuje?

V souboru `.gitlab-ci.yml` v **kořeni repozitáře**

```yaml
test:
  image: node:24
  script:
    - npm ci
    - npm test
```

Po každém `git push` GitLab tento soubor najde a pipeline **sám spustí**

První pipeline

Co se děje v kódu?

```yaml
test:
  image: node:24
  script:
    - npm ci
    - npm test
```

První pipeline

Co se děje v kódu?

```yaml
test:
  image: node:24
  script:
    - npm ci
    - npm test
```

1. `test:` – název **jobu** (úlohy)

První pipeline

Co se děje v kódu?

```yaml
test:
  image: node:24
  script:
    - npm ci
    - npm test
```

1. `test:` – název **jobu** (úlohy)
2. `image:` – kontejner, ve kterém job poběží

První pipeline

Co se děje v kódu?

```yaml
test:
  image: node:24
  script:
    - npm ci
    - npm test
```

1. `test:` – název **jobu** (úlohy)
2. `image:` – kontejner, ve kterém job poběží
3. `script:` – příkazy; když kterýkoli selže, job **spadne**

Runner

Kde se ty příkazy fyzicky spouštějí?

Runner

Kde se ty příkazy fyzicky spouštějí?

Runner je

Runner

Kde se ty příkazy fyzicky spouštějí?

Runner je stroj, který joby vykonává

Runner

Kde se ty příkazy fyzicky spouštějí?

Runner je stroj, který joby vykonává

- gitlab.com nabízí sdílené runnery

Runner

Kde se ty příkazy fyzicky spouštějí?

Runner je stroj, který joby vykonává

- gitlab.com nabízí sdílené runnery
- Školní GitLab má **vlastní** runner

Runner

Kde se ty příkazy fyzicky spouštějí?

Runner je stroj, který joby vykonává

- gitlab.com nabízí sdílené runnery
- Školní GitLab má **vlastní** runner
- Velké firmy mají celé farmy runnerů

Runner

Důležité: každý job běží v **čistém kontejneru**

Runner

Důležité: každý job běží v **čistém kontejneru**

- Žádné zbytky z minulého běhu

Runner

Důležité: každý job běží v **čistém kontejneru**

- Žádné zbytky z minulého běhu
- Žádné „u mě nainstalované“ nástroje

Runner

Důležité: každý job běží v **čistém kontejneru**

- Žádné zbytky z minulého běhu
- Žádné „u mě nainstalované“ nástroje
- Pokaždé znovu, od nuly

Runner

Důležité: každý job běží v **čistém kontejneru**

- Žádné zbytky z minulého běhu
- Žádné „u mě nainstalované“ nástroje
- Pokaždé znovu, od nuly

Právě proto je výsledek **důvěryhodný**

Runner

Otázka: proč v CI používáme `npm ci`, a ne `npm install`?

Runner

Otázka: proč v CI používáme `npm ci`, a ne `npm install`?

Odpověď: `npm ci` instaluje **přesné verze** z `package-lock.json`

Runner

Otázka: proč v CI používáme `npm ci`, a ne `npm install`?

Odpověď: `npm ci` instaluje **přesné verze** z `package-lock.json`

Vysvětlení: CI musí být **reprodukovatelné** – stejný commit musí dát stejný výsledek dnes i za měsíc

Runner

Otázka: proč v CI používáme `npm ci`, a ne `npm install`?

Odpověď: `npm ci` instaluje **přesné verze** z `package-lock.json`

Vysvětlení: CI musí být **reprodukovatelné** – stejný commit musí dát stejný výsledek dnes i za měsíc

`npm install` může verze **posunout** (a `package-lock.json` přepsat)

Stages

Skutečná pipeline má víc kroků a záleží na jejich **pořadí**

Stages

Skutečná pipeline má víc kroků a záleží na jejich **pořadí**

Stage (fáze) je

Stages

Skutečná pipeline má víc kroků a záleží na jejich **pořadí**

Stage (fáze) je skupina jobů, které běží **souběžně**

Stages

Skutečná pipeline má víc kroků a záleží na jejich **pořadí**

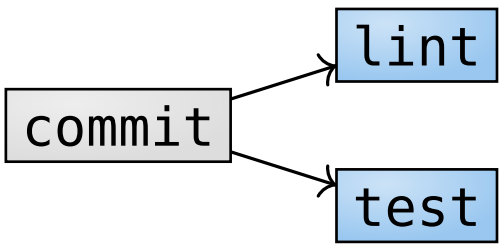
Stage (fáze) je skupina jobů, které běží **souběžně**

Další stage začne, až když **všechny joby** předchozí stage uspějí

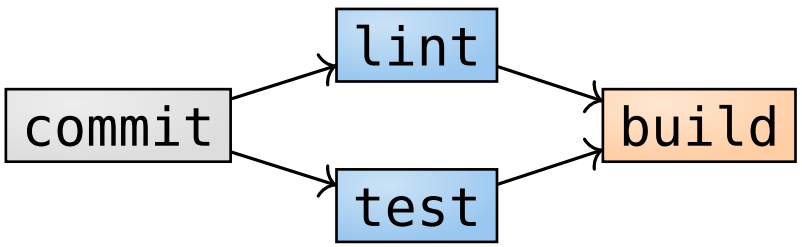
Stages

`commit`

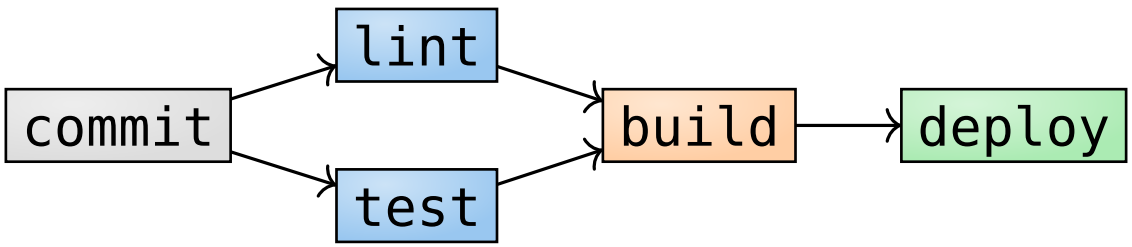
Stages



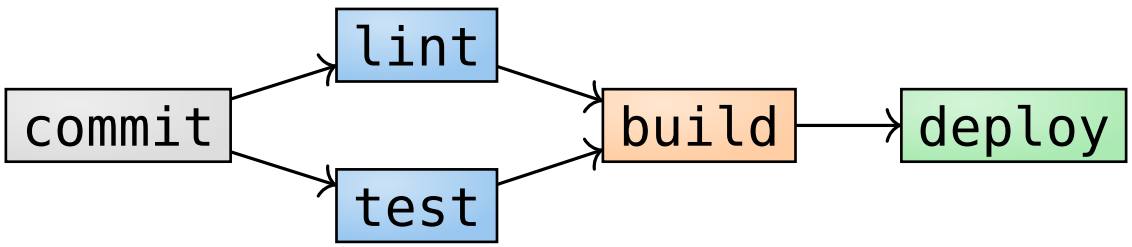
Stages



Stages

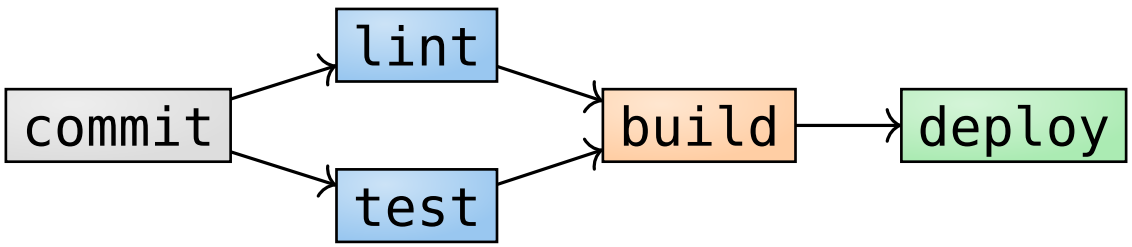


Stages



`lint` a `test` běží **souběžně**

Stages



`lint` a `test` běží **souběžně** – `build` čeká, až **oba** uspějí

Stages

V `.gitlab-ci.yml` stages nejprve **vyjmenujeme**, pak je jobům **přiřadíme**:

```yaml
stages:
  - test
  - build

lint:
  stage: test
  image: node:24
  script:
    - npm ci
    - npx eslint .
```

Stages

`test` patří do **stejné** stage jako `lint` – proto běží souběžně:

```yaml
test:
  stage: test
  image: node:24
  script:
    - npm ci
    - npm test
```

Artifacts a cache

Problém: každý job začíná v čistém kontejneru – jak dostat výsledek `build` jobu do `deploy` jobu?

Artifacts a cache

Problém: každý job začíná v čistém kontejneru – jak dostat výsledek `build` jobu do `deploy` jobu?

Artifacts – soubory, které job **uloží** a předá dál:

```yaml
build:
  script:
    - npm run build
  artifacts:
    paths:
      - dist/
```

Artifacts a cache

Problém: každý job začíná v čistém kontejneru – jak dostat výsledek `build` jobu do `deploy` jobu?

Artifacts – soubory, které job **uloží** a předá dál:

```yaml
build:
  script:
    - npm run build
  artifacts:
    paths:
      - dist/
```

Artifacts si můžete i **stáhnout** z webového rozhraní GitLabu

Artifacts a cache

A co `node_modules`? Instalovat je v každém jobu znovu je **pomalé**

Artifacts a cache

A co `node_modules`? Instalovat je v každém jobu znovu je **pomalé**

```yaml
cache:
  key:
    files:
      - package-lock.json
  paths:
    - .npm/
```

Artifacts a cache

A co `node_modules`? Instalovat je v každém jobu znovu je **pomalé**

```yaml
cache:
  key:
    files:
      - package-lock.json
  paths:
    - .npm/
```

Dokud se `package-lock.json` nezmění, balíčky se znovu nestahují

Artifacts a cache

Cache i artifacts přenášejí soubory. **V čem se liší?**

Artifacts a cache

Cache i artifacts přenášejí soubory. **V čem se liší?**

	Cache	**Artifacts**

Artifacts a cache

Cache i artifacts přenášejí soubory. **V čem se liší?**

	Cache	**Artifacts**
Účel	zrychlení buildu	výsledek práce

Artifacts a cache

Cache i artifacts přenášejí soubory. **V čem se liší?**

	Cache	**Artifacts**
Účel	zrychlení buildu	výsledek práce
Obsah	`node_modules`, `.npm`	`dist/`, reporty

Artifacts a cache

Cache i artifacts přenášejí soubory. **V čem se liší?**

	Cache	**Artifacts**
Účel	zrychlení buildu	výsledek práce
Obsah	`node_modules`, `.npm`	`dist/`, reporty
Garance	může se ztratit	uložené a stažitelné

Proměnné a secrets

Deploy potřebuje heslo k serveru. **Kam s ním?**

Proměnné a secrets

Deploy potřebuje heslo k serveru. **Kam s ním?**

```
deploy:
  script:
    - ./deploy.sh --password "hunter2"
```

Co je na tom špatně?

Proměnné a secrets

Deploy potřebuje heslo k serveru. **Kam s ním?**

```
deploy:
  script:
    - ./deploy.sh --password "hunter2"
```

Co je na tom špatně?

Heslo je **v gitu** – navždy, pro každého, kdo repozitář uvidí

Proměnné a secrets

Deploy potřebuje heslo k serveru. **Kam s ním?**

```
deploy:
  script:
    - ./deploy.sh --password "hunter2"
```

Co je na tom špatně?

Heslo je **v gitu** – navždy, pro každého, kdo repozitář uvidí

Nikdy necommitujte tajemství do repozitáře!

Proměnné a secrets

Řešení: CI/CD Variables v nastavení projektu

Proměnné a secrets

Řešení: CI/CD Variables v nastavení projektu

- Settings → CI/CD → Variables

Proměnné a secrets

Řešení: CI/CD Variables v nastavení projektu

- Settings → CI/CD → Variables
- **Masked** – hodnota se nezobrazí v logu

Proměnné a secrets

Řešení: CI/CD Variables v nastavení projektu

- Settings → CI/CD → Variables
- **Masked** – hodnota se nezobrazí v logu
- **Protected** – dostupná jen na chráněných větvích

Proměnné a secrets

Řešení: CI/CD Variables v nastavení projektu

- Settings → CI/CD → Variables
- **Masked** – hodnota se nezobrazí v logu
- **Protected** – dostupná jen na chráněných větvích

V pipeline pak proměnnou jen použijete:

```
deploy:
  script:
    - ./deploy.sh --password "$DEPLOY_PASSWORD"
```

Proměnné a secrets

GitLab navíc řadu proměnných **předdefinuje**:

Proměnné a secrets

GitLab navíc řadu proměnných **předdefinuje**:

Proměnná	Obsah

Proměnné a secrets

GitLab navíc řadu proměnných **předdefinuje**:

Proměnná	Obsah
`CI_COMMIT_SHORT_SHA`	zkrácený hash commitu

Proměnné a secrets

GitLab navíc řadu proměnných **předdefinuje**:

Proměnná	Obsah
`CI_COMMIT_SHORT_SHA`	zkrácený hash commitu
`CI_COMMIT_BRANCH`	název větve

Proměnné a secrets

GitLab navíc řadu proměnných **předdefinuje**:

Proměnná	Obsah
`CI_COMMIT_SHORT_SHA`	zkrácený hash commitu
`CI_COMMIT_BRANCH`	název větve
`CI_REGISTRY_IMAGE`	adresa image v registry projektu

Proměnné a secrets

GitLab navíc řadu proměnných **předdefinuje**:

Proměnná	Obsah
`CI_COMMIT_SHORT_SHA`	zkrácený hash commitu
`CI_COMMIT_BRANCH`	název větve
`CI_REGISTRY_IMAGE`	adresa image v registry projektu

Za chvíli je využijeme pro stavbu Docker image

Pipeline a Merge Requesty

Vzpomeňte si na přednášku o kolaborativním vývoji

Pipeline a Merge Requesty

Vzpomeňte si na přednášku o kolaborativním vývoji

„CI/CD pipeline je zelená = build a testy procházejí“

Pipeline a Merge Requesty

Vzpomeňte si na přednášku o kolaborativním vývoji

„CI/CD pipeline je zelená = build a testy procházejí“

GitLab umí nastavit, že MR **nejde mergnout**, dokud pipeline neprojde

Pipeline a Merge Requesty

Vzpomeňte si na přednášku o kolaborativním vývoji

„CI/CD pipeline je zelená = build a testy procházejí“

GitLab umí nastavit, že MR **nejde mergnout**, dokud pipeline neprojde

- Reviewer čte kód s jistotou, že **funguje**

Pipeline a Merge Requesty

Vzpomeňte si na přednášku o kolaborativním vývoji

„CI/CD pipeline je zelená = build a testy procházejí“

GitLab umí nastavit, že MR **nejde mergnout**, dokud pipeline neprojde

- Reviewer čte kód s jistotou, že **funguje**
- Do `main` se nedostane nic rozbitého

Pipeline a Merge Requesty

Vzpomeňte si na přednášku o kolaborativním vývoji

„CI/CD pipeline je zelená = build a testy procházejí“

GitLab umí nastavit, že MR **nejde mergnout**, dokud pipeline neprojde

- Reviewer čte kód s jistotou, že **funguje**
- Do `main` se nedostane nic rozbitého

Co neprojde pipeline,

Pipeline a Merge Requesty

Vzpomeňte si na přednášku o kolaborativním vývoji

„CI/CD pipeline je zelená = build a testy procházejí“

GitLab umí nastavit, že MR **nejde mergnout**, dokud pipeline neprojde

- Reviewer čte kód s jistotou, že **funguje**
- Do `main` se nedostane nic rozbitého

Co neprojde pipeline, to nejde do `main`!

Pipeline pro naši aplikaci

Lint, testy a build

Vraťme se k Express aplikaci z minulých přednášek a postavme jí **kompletní pipeline**

Lint, testy a build

Vraťme se k Express aplikaci z minulých přednášek a postavme jí **kompletní pipeline**

Stage `test` už umíme – joby `lint` a `test` z první poloviny přednášky

Lint, testy a build

Vraťme se k Express aplikaci z minulých přednášek a postavme jí **kompletní pipeline**

Stage `test` už umíme – joby `lint` a `test` z první poloviny přednášky

Zbývá stage `build`:

Lint, testy a build

Vraťme se k Express aplikaci z minulých přednášek a postavme jí **kompletní pipeline**

Stage `test` už umíme – joby `lint` a `test` z první poloviny přednášky

Zbývá stage `build`: přeložit TypeScript a postavit Docker image

Lint, testy a build

Překlad TypeScriptu:

```yaml
build:
  stage: build
  image: node:24
  script:
    - npm ci
    - npm run build
  artifacts:
    paths:
      - dist/
```

Lint, testy a build

Překlad TypeScriptu:

```yaml
build:
  stage: build
  image: node:24
  script:
    - npm ci
    - npm run build
  artifacts:
    paths:
      - dist/
```

Tím máme jistotu, že se projekt **dá přeložit**

Lint, testy a build

Překlad TypeScriptu:

```yaml
build:
  stage: build
  image: node:24
  script:
    - npm ci
    - npm run build
  artifacts:
    paths:
      - dist/
```

Tím máme jistotu, že se projekt **dá přeložit** – a přeložený kód poputuje dál

Docker image v CI

Minule jsme pro aplikaci napsali `Dockerfile` a stavěli image **ručně**

Docker image v CI

Minule jsme pro aplikaci napsali `Dockerfile` a stavěli image **ručně**

Teď to za nás udělá pipeline:

```yaml
docker-build:
  stage: build
  image: docker:27
  services:
    - docker:27-dind
  script:
    - docker login -u "$CI_REGISTRY_USER"
        -p "$CI_REGISTRY_PASSWORD" "$CI_REGISTRY"
    - docker build
        -t "$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA" .
    - docker push
        "$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA"
```

Docker image v CI

Co se děje v kódu?

Docker image v CI

Co se děje v kódu?

1. `services: docker:27-dind` –

Docker image v CI

Co se děje v kódu?

1. `services: docker:27-dind` – uvnitř jobu běží **Docker in Docker**

Docker image v CI

Co se děje v kódu?

1. `services: docker:27-dind` – uvnitř jobu běží **Docker in Docker**
2. `docker login` – **GitLab Container Registry**, Docker Hub zabudovaný v GitLabu

Docker image v CI

Co se děje v kódu?

1. `services: docker:27-dind` – uvnitř jobu běží **Docker in Docker**
2. `docker login` – **GitLab Container Registry**, Docker Hub zabudovaný v GitLabu
3. `docker build` – image otagovaný **hashem commitu**

Docker image v CI

Co se děje v kódu?

1. `services: docker:27-dind` – uvnitř jobu běží **Docker in Docker**
2. `docker login` – **GitLab Container Registry**, Docker Hub zabudovaný v GitLabu
3. `docker build` – image otagovaný **hashem commitu** – vždy víme, jaká verze kódu v něm je

Docker image v CI

Co se děje v kódu?

1. `services: docker:27-dind` – uvnitř jobu běží **Docker in Docker**
2. `docker login` – **GitLab Container Registry**, Docker Hub zabudovaný v GitLabu
3. `docker build` – image otagovaný **hashem commitu** – vždy víme, jaká verze kódu v něm je
4. `docker push` – image je v registry, připravený k nasazení

Pravidla spouštění

Chceme stavět image z **každé** větve?

Pravidla spouštění

Chceme stavět image z **každé** větve?

Ne – image nasazujeme jen z otestované hlavní větve

Pravidla spouštění

Chceme stavět image z **každé** větve?

Ne – image nasazujeme jen z otestované hlavní větve

```
docker-build:
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Pravidla spouštění

Chceme stavět image z **každé** větve?

Ne – image nasazujeme jen z otestované hlavní větve

```
docker-build:
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

- na `main` poběží lint, testy, build **i Docker image**

Pravidla spouštění

Chceme stavět image z **každé** větve?

Ne – image nasazujeme jen z otestované hlavní větve

```
docker-build:
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

- na `main` poběží lint, testy, build **i Docker image**
- na feature větvích jen lint, testy a build

Debugging pipeline

Červená pipeline

Pipeline spadla. **Co teď?**

Červená pipeline

Pipeline spadla. **Co teď?**

Častá reakce: „Spustím to znovu, třeba to projde“

Červená pipeline

Pipeline spadla. **Co teď?**

Častá reakce: „Spustím to znovu, třeba to projde“

NE!

Červená pipeline

Pipeline spadla. **Co teď?**

Častá reakce: „Spustím to znovu, třeba to projde“

NE! Pipeline je deterministická – stejný kód dá stejný výsledek

Červená pipeline

Pipeline spadla. **Co teď?**

Častá reakce: „Spustím to znovu, třeba to projde“

NE! Pipeline je deterministická – stejný kód dá stejný výsledek

Správný postup:

Červená pipeline

Pipeline spadla. **Co teď?**

Častá reakce: „Spustím to znovu, třeba to projde“

NE! Pipeline je deterministická – stejný kód dá stejný výsledek

Správný postup: otevřít **log jobu** a najít **první chybu**

Červená pipeline

Pipeline spadla. **Co teď?**

Častá reakce: „Spustím to znovu, třeba to projde“

NE! Pipeline je deterministická – stejný kód dá stejný výsledek

Správný postup: otevřít **log jobu** a najít **první chybu** – vše za ní bývá jen následek

Nejčastější příčiny

Co pipeline shazuje nejčastěji (nejen u domácích úkolů):

Nejčastější příčiny

Co pipeline shazuje nejčastěji (nejen u domácích úkolů):

- **Lint chyby** – lokálně jste lint nespustili

Nejčastější příčiny

Co pipeline shazuje nejčastěji (nejen u domácích úkolů):

- **Lint chyby** – lokálně jste lint nespustili
- **Failing test** – změna rozbila něco jinde

Nejčastější příčiny

Co pipeline shazuje nejčastěji (nejen u domácích úkolů):

- **Lint chyby** – lokálně jste lint nespustili
- **Failing test** – změna rozbila něco jinde
- `npm ci` selže – `package-lock.json` nesedí s `package.json`

Nejčastější příčiny

Co pipeline shazuje nejčastěji (nejen u domácích úkolů):

- **Lint chyby** – lokálně jste lint nespustili
- **Failing test** – změna rozbila něco jinde
- `npm ci` selže – `package-lock.json` nesedí s `package.json`
- Lokálně **jiná verze Node.js** než v `image:`

Nejčastější příčiny

Co pipeline shazuje nejčastěji (nejen u domácích úkolů):

- **Lint chyby** – lokálně jste lint nespustili
- **Failing test** – změna rozbila něco jinde
- `npm ci` selže – `package-lock.json` nesedí s `package.json`
- Lokálně **jiná verze Node.js** než v `image:`
- Soubor jste **zapomněli commitnout** – lokálně existuje, v CI ne

Nejčastější příčiny

Co pipeline shazuje nejčastěji (nejen u domácích úkolů):

- **Lint chyby** – lokálně jste lint nespustili
- **Failing test** – změna rozbila něco jinde
- `npm ci` selže – `package-lock.json` nesedí s `package.json`
- Lokálně **jiná verze Node.js** než v `image:`
- Soubor jste **zapomněli commitnout** – lokálně existuje, v CI ne

Všimněte si:

Nejčastější příčiny

Co pipeline shazuje nejčastěji (nejen u domácích úkolů):

- **Lint chyby** – lokálně jste lint nespustili
- **Failing test** – změna rozbila něco jinde
- `npm ci` selže – `package-lock.json` nesedí s `package.json`
- Lokálně **jiná verze Node.js** než v `image:`
- Soubor jste **zapomněli commitnout** – lokálně existuje, v CI ne

Všimněte si: pipeline vždy jen **dřív odhalila** skutečný problém

Continuous Delivery a Deployment

Definice CD

CI ověřilo každý commit. Co s otestovanou verzí dál?

Definice CD

CI ověřilo každý commit. Co s otestovanou verzí dál?

Continuous Delivery (průběžné doručování) znamená, že

Definice CD

CI ověřilo každý commit. Co s otestovanou verzí dál?

Continuous Delivery (průběžné doručování) znamená, že každá zelená verze je **připravená k nasazení** – nasazení spouští člověk tlačítkem

Definice CD

CI ověřilo každý commit. Co s otestovanou verzí dál?

Continuous Delivery (průběžné doručování) znamená, že každá zelená verze je **připravená k nasazení** – nasazení spouští člověk tlačítkem

Continuous Deployment jde ještě dál:

Definice CD

CI ověřilo každý commit. Co s otestovanou verzí dál?

Continuous Delivery (průběžné doručování) znamená, že každá zelená verze je **připravená k nasazení** – nasazení spouští člověk tlačítkem

Continuous Deployment jde ještě dál: nasazení proběhne **automaticky**, bez člověka

Definice CD

CI ověřilo každý commit. Co s otestovanou verzí dál?

Continuous Delivery (průběžné doručování) znamená, že každá zelená verze je **připravená k nasazení** – nasazení spouští člověk tlačítkem

Continuous Deployment jde ještě dál: nasazení proběhne **automaticky**, bez člověka

Pozn.: Pojem rozšířila kniha Continuous Delivery (Jez Humble, David Farley, 2010)

Definice CD

	Delivery	Deployment

Definice CD

	Delivery	Deployment
Testy a build	automaticky	automaticky

Definice CD

	Delivery	Deployment
Testy a build	automaticky	automaticky
Nasazení	člověk klikne	automaticky

Definice CD

	Delivery	Deployment
Testy a build	automaticky	automaticky
Nasazení	člověk klikne	automaticky
Hodí se	většina firem	zralé týmy s monitoringem

Definice CD

	Delivery	Deployment
Testy a build	automaticky	automaticky
Nasazení	člověk klikne	automaticky
Hodí se	většina firem	zralé týmy s monitoringem

Obojí zkráceně **CD** – proto se říká CI/CD

Deploy stage

Poslední stage naší pipeline:

```yaml
deploy:
  stage: deploy
  script:
    - ./deploy.sh
  environment: production
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual
```

Deploy stage

Poslední stage naší pipeline:

```yaml
deploy:
  stage: deploy
  script:
    - ./deploy.sh
  environment: production
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual
```

`when: manual` – nasazení spustí člověk tlačítkem

Deploy stage

Poslední stage naší pipeline:

```yaml
deploy:
  stage: deploy
  script:
    - ./deploy.sh
  environment: production
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual
```

`when: manual` – nasazení spustí člověk tlačítkem (Continuous **Delivery**)

Deploy stage

Co je uvnitř `deploy.sh` a kam se vlastně nasazuje?

Deploy stage

Co je uvnitř `deploy.sh` a kam se vlastně nasazuje?

To si ukážeme na příští přednášce

Deploy stage

Co je uvnitř `deploy.sh` a kam se vlastně nasazuje?

To si ukážeme na příští přednášce

Dnes nám stačí vědět:

Deploy stage

Co je uvnitř `deploy.sh` a kam se vlastně nasazuje?

To si ukážeme na příští přednášce

Dnes nám stačí vědět: po kliknutí se otestovaná verze dostane k uživatelům

CI/CD v praxi

Nástroje

GitLab CI není jediný:

Nástroje

GitLab CI není jediný:

Nástroj	Konfigurace	Kde se s ním potkáte

Nástroje

GitLab CI není jediný:

Nástroj	Konfigurace	Kde se s ním potkáte
GitLab CI	`.gitlab-ci.yml`	firmy s vlastním GitLabem

Nástroje

GitLab CI není jediný:

Nástroj	Konfigurace	Kde se s ním potkáte
GitLab CI	`.gitlab-ci.yml`	firmy s vlastním GitLabem
GitHub Actions	`.github/workflows/`	open source, startupy

Nástroje

GitLab CI není jediný:

Nástroj	Konfigurace	Kde se s ním potkáte
GitLab CI	`.gitlab-ci.yml`	firmy s vlastním GitLabem
GitHub Actions	`.github/workflows/`	open source, startupy
Jenkins (2011)	`Jenkinsfile`	korporace, starší systémy

Nástroje

GitLab CI není jediný:

Nástroj	Konfigurace	Kde se s ním potkáte
GitLab CI	`.gitlab-ci.yml`	firmy s vlastním GitLabem
GitHub Actions	`.github/workflows/`	open source, startupy
Jenkins (2011)	`Jenkinsfile`	korporace, starší systémy

Principy jsou **stejné** – joby, stages, runnery – mění se jen syntaxe

Nástroje

Zajímavost

Nástroje

Zajímavost

Slidy tohoto předmětu kompiluje GitHub Actions

Nástroje

Zajímavost

Slidy tohoto předmětu kompiluje GitHub Actions

Každý push přeloží všechny přednášky do PDF

Nástroje

Zajímavost

Slidy tohoto předmětu kompiluje GitHub Actions

Každý push přeloží všechny přednášky do PDF – i tuhle

Dopad na vývoj

Vyplatí se ta práce s nastavováním?

Dopad na vývoj

Vyplatí se ta práce s nastavováním?

Výzkum DORA měří výkonnost tisíců vývojových týmů:

Dopad na vývoj

Vyplatí se ta práce s nastavováním?

Výzkum DORA měří výkonnost tisíců vývojových týmů:

- Elitní týmy nasazují **několikrát denně**

Dopad na vývoj

Vyplatí se ta práce s nastavováním?

Výzkum DORA měří výkonnost tisíců vývojových týmů:

- Elitní týmy nasazují **několikrát denně**
- Změna se k uživatelům dostane **do hodiny**

Dopad na vývoj

Vyplatí se ta práce s nastavováním?

Výzkum DORA měří výkonnost tisíců vývojových týmů:

- Elitní týmy nasazují **několikrát denně**
- Změna se k uživatelům dostane **do hodiny**
- A přitom mají **méně** rozbitých nasazení než ostatní

Dopad na vývoj

Vyplatí se ta práce s nastavováním?

Výzkum DORA měří výkonnost tisíců vývojových týmů:

- Elitní týmy nasazují **několikrát denně**
- Změna se k uživatelům dostane **do hodiny**
- A přitom mají **méně** rozbitých nasazení než ostatní

Rychlost a spolehlivost **nejsou protiklady** – automatizace přináší obojí

Dopad na vývoj

Vyplatí se ta práce s nastavováním?

Výzkum DORA měří výkonnost tisíců vývojových týmů:

- Elitní týmy nasazují **několikrát denně**
- Změna se k uživatelům dostane **do hodiny**
- A přitom mají **méně** rozbitých nasazení než ostatní

Rychlost a spolehlivost **nejsou protiklady** – automatizace přináší obojí

Zdroj: DORA – State of DevOps Report (Google Cloud)

„Funguje mi to lokálně“ nikoho nezajímá

Musí to projít pipeline!

Shrnutí

Klíčové poznatky

Co jsme se dnes naučili:

Klíčové poznatky

Co jsme se dnes naučili:

- **CI** = každou změnu automaticky ověří stroj – lint, testy, build

Klíčové poznatky

Co jsme se dnes naučili:

- **CI** = každou změnu automaticky ověří stroj – lint, testy, build
- Pipeline se definuje v `.gitlab-ci.yml`, vykonávají ji **runnery** v čistých kontejnerech

Klíčové poznatky

Co jsme se dnes naučili:

- **CI** = každou změnu automaticky ověří stroj – lint, testy, build
- Pipeline se definuje v `.gitlab-ci.yml`, vykonávají ji **runnery** v čistých kontejnerech
- **Stages** řadí joby za sebe, **artifacts** předávají výsledky, **cache** zrychluje

Klíčové poznatky

Co jsme se dnes naučili:

- **CI** = každou změnu automaticky ověří stroj – lint, testy, build
- Pipeline se definuje v `.gitlab-ci.yml`, vykonávají ji **runnery** v čistých kontejnerech
- **Stages** řadí joby za sebe, **artifacts** předávají výsledky, **cache** zrychluje
- Tajemství patří do **CI/CD Variables**, nikdy do gitu

Klíčové poznatky

Co jsme se dnes naučili:

- **CI** = každou změnu automaticky ověří stroj – lint, testy, build
- Pipeline se definuje v `.gitlab-ci.yml`, vykonávají ji **runnery** v čistých kontejnerech
- **Stages** řadí joby za sebe, **artifacts** předávají výsledky, **cache** zrychluje
- Tajemství patří do **CI/CD Variables**, nikdy do gitu
- **CD** = otestovaná verze je vždy připravená k nasazení

Klíčové poznatky

Vyzkoušejte si sami:

Klíčové poznatky

Vyzkoušejte si sami:

- Otevřete `.gitlab-ci.yml` ve svém domácím úkolu – teď už mu rozumíte

Klíčové poznatky

Vyzkoušejte si sami:

- Otevřete `.gitlab-ci.yml` ve svém domácím úkolu – teď už mu rozumíte
- GitLab má vestavěný **pipeline editor** s validací (Build → Pipeline editor)

Klíčové poznatky

Vyzkoušejte si sami:

- Otevřete `.gitlab-ci.yml` ve svém domácím úkolu – teď už mu rozumíte
- GitLab má vestavěný **pipeline editor** s validací (Build → Pipeline editor)
- Zkuste do úkolu přidat vlastní job – třeba `npm audit`

Klíčové poznatky

Vyzkoušejte si sami:

- Otevřete `.gitlab-ci.yml` ve svém domácím úkolu – teď už mu rozumíte
- GitLab má vestavěný **pipeline editor** s validací (Build → Pipeline editor)
- Zkuste do úkolu přidat vlastní job – třeba `npm audit`

Kam pipeline nasazuje a co se s aplikací děje pak, nás čeká **příště**

Užitečné zdroje

Dokumentace

- GitLab CI/CD: https://docs.gitlab.com/ee/ci/
- GitHub Actions: https://docs.github.com/en/actions

Užitečné zdroje

Dokumentace

- GitLab CI/CD: https://docs.gitlab.com/ee/ci/
- GitHub Actions: https://docs.github.com/en/actions

Doporučená četba

- Continuous Delivery (Jez Humble, David Farley)
- DORA – State of DevOps Report: https://dora.dev/